Projet Agent - Compte Rendu

MOREAU Wallis
22603080
wallis.moreau@etu.umontpellier.fr

PRATVIEL Sébastien
22617171
sebastien.pratviel@etu.umontpellier.fr

Vendredi 30/10/2026

Partie 1 - Architecture

Schéma

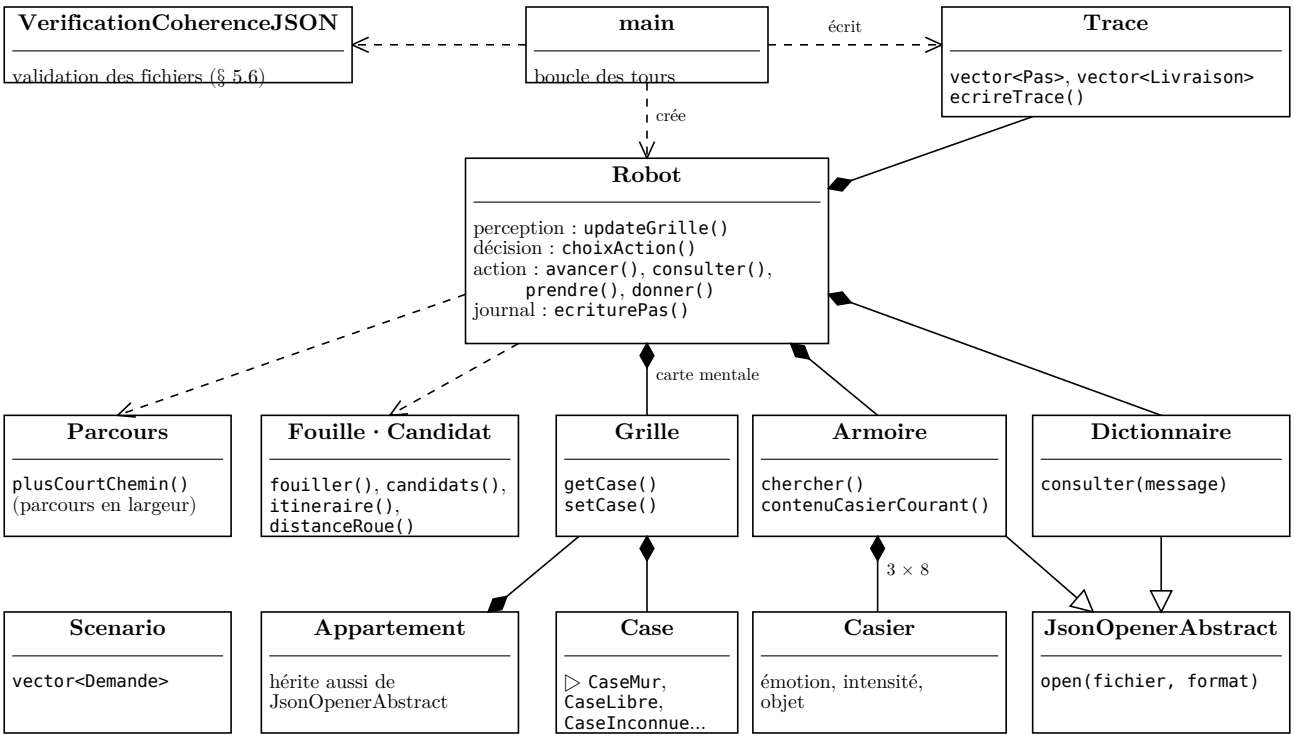


Fig. 1. – Diagramme de classes simplifié. ◆ composition, ▷ héritage, flèche pointillée : dépendance (« utilise »). Le robot possède sa carte mentale et garde un pointeur constant vers la vraie grille.

Justification des frontières

Le robot est une classe `Robot` dont les méthodes correspondent aux quatre rôles d'un agent. Les algorithmes « purs », qui ne dépendent d'aucun état, sont sortis dans des fichiers séparés pour être testables seuls.

- **Perception** : `updateGrille()` lit les quatre cases voisines du robot dans la vraie grille et les inscrit dans la carte mentale. C'est le seul endroit où le robot accède à la vraie carte, et il n'en voit que ces quatre cases.
- **Mémoire** : la carte mentale est une seconde `Grille`, distincte de la vraie, dont toutes les cases intérieures sont initialisées à « inconnu » (`?`). Seuls les bords, l'armoire, le dictionnaire et les résidents sont connus d'avance. S'y ajoutent la position, l'objet porté, l'émotion de la demande en cours et l'avancement de la fouille.
- **Décision** : `choixAction()` choisit l'action du tour selon l'étape en cours (dictionnaire, armoire ou résident) et selon que le robot est adjacent ou non à sa cible. Elle s'appuie sur `plusCourtChemin()` (`Parcours.hpp`) pour se déplacer et sur `candidats()` / `fouiller()` (`Candidat.hpp`, `Fouille.hpp`) pour choisir le casier.

- **Action** : `avancer()`, `consulter()`, `prendre()` et `donner()` exécutent l'action, et chacune est journalisée par `ecriturePas()` **avant** d'être exécutée, puisque la trace demande la position et le sélecteur d'avant l'action.

Les structures demandées sont écrites à la main : le parcours en largeur utilise une file (`std::queue`), et le caractère cyclique de la roue et des colonnes de l'armoire est géré par un modulo 8 (`(c + 8) % 8`, le `%` du C++ pouvant donner un résultat négatif).

Limites. La frontière n'est pas parfaite : la perception est faite par le robot lui-même, qui garde un pointeur vers la vraie grille. Un simulateur séparé, qui fournirait les perceptions au robot, rendrait la séparation plus nette, et le robot pourrait être totalement indépendant de la grille initiale.

Partie 2 - Stratégie de replanification

Le robot **replanifie entièrement à chaque tour**. À chaque appel de `choixAction()`, s'il n'est pas adjacent à sa cible, il relance un parcours en largeur depuis sa position sur la carte mentale, avec une hypothèse optimiste : une case inconnue est considérée comme traversable. Il n'exécute que le premier pas du chemin obtenu, perçoit, met à jour sa carte, puis recommence au tour suivant. Quand un mur est découvert sur le chemin prévu, il n'y a donc rien à « réparer » : le calcul suivant en tient compte naturellement.

La cible n'est jamais une case unique : l'armoire, le dictionnaire et les résidents ne sont pas traversables, donc le parcours s'arrête sur la première de leurs quatre cases voisines atteinte, qui est la plus proche grâce à l'ordre de la file.

Coût. Un parcours en largeur coûte $O(H \times L)$ par tour, soit au plus 273 cases sur la plus grande carte fournie (13×21). Nous pensons que cela est assez négligeable.

Exemple. Sur la carte 01, le robot qui part vers Elodie en suivant la ligne 6 découvre le mur `[6,7]` en arrivant en `[6,6]` ; le parcours suivant le fait remonter par la ligne 2 jusqu'à `[4,11]`, seule entrée de la partie droite.

Inaccessibilité. Si le parcours échoue alors que toutes les cases inconnues sont supposées libres, aucun chemin ne peut exister dans la réalité : c'est la preuve qu'un résident est inaccessible. Le robot enregistre alors l'échec et passe à la demande suivante.

Partie 3 - Critique sur la règle de repli

La règle impose, quand le casier exact est vide, d'essayer la même émotion à une autre intensité, puis les émotions voisines sur la roue de Plutchik à distance 1, puis 2. Elle a une grande qualité : elle privilégie, à raison, la bonne émotion sur la bonne intensité.

Elle repose pourtant sur une confusion. La roue de Plutchik mesure une proximité entre émotions, pas une proximité entre besoins. Deux émotions voisines peuvent appeler des réponses opposées. Nos traces en donnent un exemple concret : sur la carte 03, un résident « sidéré » parce que sa télévision s'est allumée seule reçoit un téléphone, rangé sous la tristesse. L'objet n'a aucun rapport avec ce qu'il ressent.

Le problème s'aggrave si l'on se rapproche de la réalité, où un objet donné disparaît de l'armoire. Donner ce téléphone serait alors doublement contre-productif : il ne répond pas à la demande, et si le résident suivant est vraiment triste et a besoin d'appeler un proche, le téléphone n'est plus disponible. La règle actuelle décide demande par demande, sans tenir compte des besoins qui vont suivre.

Proposition. Nous proposons deux changements complémentaires.

1. **Décrire les objets par les émotions qu'ils apaisent.** Dans `armoire.json`, chaque casier recevrait un champ `apaise` : une liste de couples émotion / poids (par exemple, la couverture apaise la tristesse avec un poids 1 et la peur avec un poids 0,6). Le score d'un objet pour une demande serait le poids de l'émotion demandée, l'intensité servant à départager. La fonction `candidats()` trierait les casiers par score au lieu de la distance sur la roue.
2. **Répartir les objets sur l'ensemble des demandes.** Avec un stock fini (un champ `quantite` par casier dans `armoire.json`), le robot commencerait par déterminer l'émotion de toutes les demandes

du scénario, puis attribuerait les objets de façon à maximiser le score total : un objet n'est donné en repli que si aucune demande à venir n'en a davantage besoin.

Côté fichiers, le dictionnaire et le scénario ne changent pas ; seule l'armoire gagne les champs `apaise` et `quantite`, et l'étiquette `repli` de la trace deviendrait un score plutôt qu'une distance.

Partie 4 - 3 cas d'échecs du robot

1. **Zigzags quand un mur est découvert case par case.**
   Sur la carte 05, le robot fait plusieurs allers-retours avant de trouver un passage. **Cause** : un long mur n'est découvert qu'une case à la fois, et comme les cases inconnues sont supposées libres, le robot essaie chaque passage l'un après l'autre. Le trajet est correct, mais le robot fait plus de pas qu'il ne pourrais/devrais en faire.
2. **Objet perdu quand le résident est inaccessible.**
   Le robot prend l'objet avant de découvrir que le résident est muré. **Cause** : l'ordre imposé (dictionnaire, armoire, puis résident) et l'absence d'action pour reposer un objet. Le robot l'abandonne en passant à la demande suivante. Idéalement il faudrait peut-être que le robot vérifie que le résident est accessible.
3. **Sélecteur en avance pendant la fouille.**
   La fouille est calculée d'un coup au premier tour devant l'armoire, et le sélecteur réel est déplacé immédiatement jusqu'au casier final. Cela va donc forcer à avoir un attribut **selecteurTrace** dans le robot pour que la trace puisse s'écrire sélecteur par sélecteur lors de la fouille.